Assignment 2: Model Free Reinforcement Learning

Sertan Erdogan (s5152283) & Hossein Kelisa (s5152135)
Group Number: 18

1 Introduction

This report investigates four model-free reinforcement learning algorithms — Q-Learning, SARSA, Expected SARSA, and n-step SARSA — in the ShortCut environment: a 12×12 grid where an agent navigates from one of two starting positions to a goal while avoiding a cliff region. Every step carries a reward of -1; falling into the cliff yields -100. The agent maximizes the cumulative reward per episode, which corresponds to finding the fastest safe path.

All algorithms maintain a Q-table $Q(s, a)$, initialized to zero, which stores estimated future rewards for each state-action pair. Actions are selected via an ε-greedy policy: with probability $\varepsilon = 0.1$ a random action is taken (exploration), otherwise the action with the highest Q-value is chosen (exploitation). We use $\gamma = 1.0$ throughout. All learning curves are averaged over 100 independent repetitions, each with 1000 episodes. A second variant, `WindyShortcutEnvironment`, adds a 50% chance of being pushed one square downward after every action, making the environment stochastic.

2 Q-Learning

2.1 Methodology

Q-Learning is an off-policy temporal difference algorithm: it learns about the optimal greedy policy while following an exploratory one. The update rule [1, Eq. 6.8] is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\left[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right] \tag{1}$$

The term in brackets is the TD error, measuring how wrong the current estimate was. The learning rate α controls how strongly this error corrects $Q(s, a)$. The defining feature of Q-Learning is the max operator: it bootstraps from the best possible next action, regardless of what will actually be taken, making the algorithm optimistic about future behavior.

2.2 Results

After 10 000 episodes with $\alpha = \varepsilon = 0.1$, the greedy policy (Figure 1) follows a path that passes close to the cliff. Because updates assume optimal future behavior, the algorithm never accounts for the risk of accidental exploration steps near the cliff — the short path looks optimal from Q-Learning's perspective.

The averaged learning curve (Figure 2) rises steadily as the Q-table improves. Early episodes accumulate very negative rewards due to frequent cliff falls; reward stabilizes as the agent finds consistent paths.

Figure 3 shows the effect of $\alpha \in \{0.01, 0.1, 0.5, 0.9\}$. With $\alpha = 0.01$ the curve barely rises within 1000 episodes, as each experience changes Q-values by only 1%. With $\alpha = 0.9$ convergence is fast but unstable, as each update nearly overwrites the previous belief. The value $\alpha = 0.1$ offers a reliable balance between speed and stability.

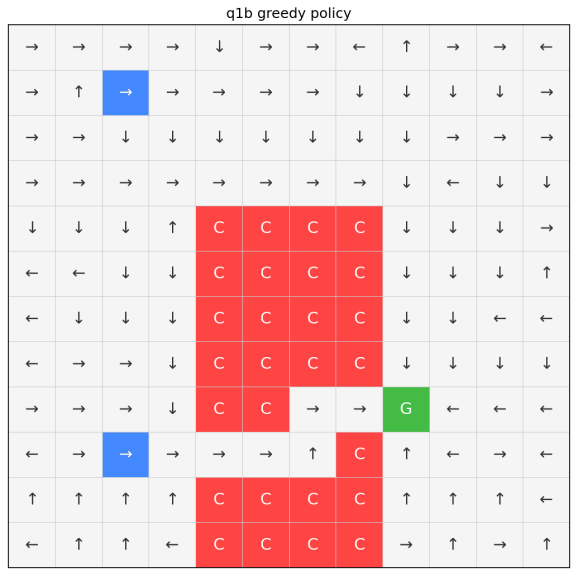


Figure 1: Greedy policy of Q-Learning after 10 000 episodes. The path passes close to the cliff (C squares).

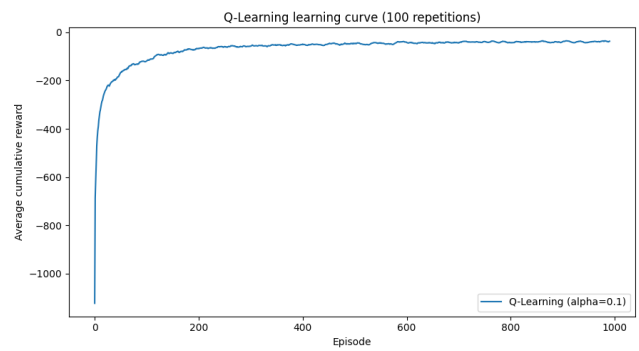


Figure 2: Average cumulative reward per episode for Q-Learning ($\alpha = 0.1$, $\varepsilon = 0.1$), averaged over 100 repetitions.

2.3 Interpretation

The α parameter encodes how much a single new experience should change current beliefs. Very small values treat each experience as nearly irrelevant; very large values treat it as decisive, speeding early learning but preventing stable convergence. Q-Learning's cliff-adjacent path is a useful illustration: the algorithm finds the objectively shortest route, but that route is only safe if the agent never explores randomly near the cliff edge — which during training it always does.

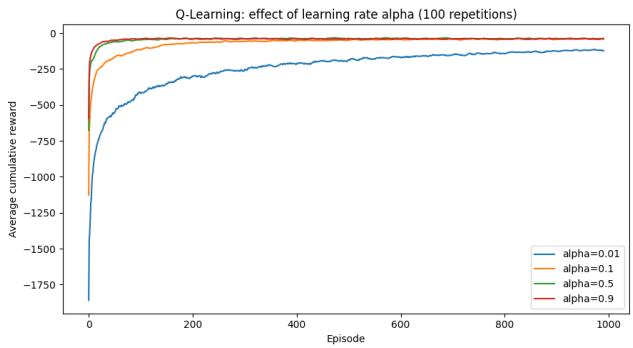


Figure 3: Q-Learning learning curves for $\alpha \in \{0.01, 0.1, 0.5, 0.9\}$, 100 repetitions.

3 SARSA

3.1 Methodology

SARSA is an on-policy temporal difference algorithm: it learns about the policy it is actually executing, including its exploratory randomness. The update rule [1, Eq. 6.7] is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma\, Q(s',a') - Q(s,a)\big] \tag{2}$$

The only difference from Equation 1 is that $Q(s',a')$ replaces $\max_{a'} Q(s',a')$: SARSA uses the actual next action a' that the agent commits to under its current ε-greedy policy. This means Q-values reflect the true cost of each state, given that the agent sometimes takes random actions. The name SARSA refers to the five elements of each update: State, Action, Reward, next State, next Action.

3.2 Results

SARSA's greedy policy (Figure 4) stays further from the cliff than Q-Learning's. Every time the agent fell off the cliff during training, SARSA incorporated that event into nearby Q-values, gradually making those states less attractive. This reproduces the classic Cliff Walking result from Sutton & Barto [1, p. 132].

As shown in Figure 5, the averaged learning curves for SARSA and Q-Learning are nearly identical in the deterministic environment. Once the policy is nearly greedy, the difference between $Q(s',a')$ and $\max Q(s',a')$ becomes small, so performance converges to the same level. The α comparison (Figure 6) follows the same pattern as Q-Learning, confirming that the speed-stability trade-off is a universal property of gradient-style learning, not specific to any one algorithm.

3.3 Interpretation

Similar learning curves but visibly different greedy paths capture the core distinction between on-policy and off-policy methods. Q-Learning learns about the best theoretically possible behavior; SARSA learns about the actual behavior of the agent, exploration included. In a safe environment, this distinction is minor. In a dangerous one, it becomes critical, as the next section demonstrates.

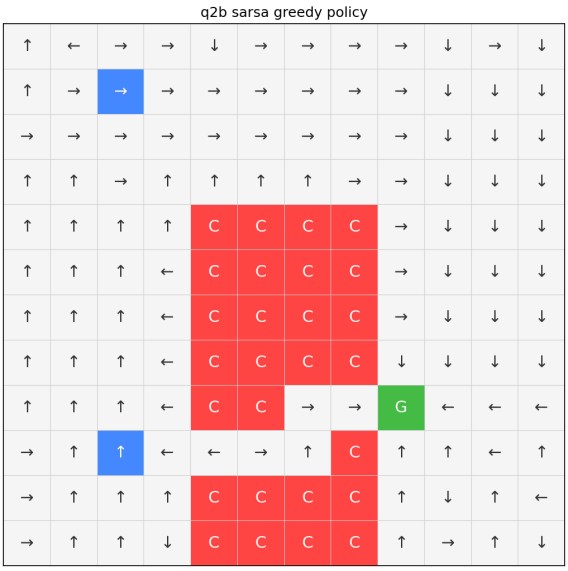


Figure 4: Greedy policy of SARSA after 10 000 episodes. The path avoids the cliff more cautiously than Q-Learning.

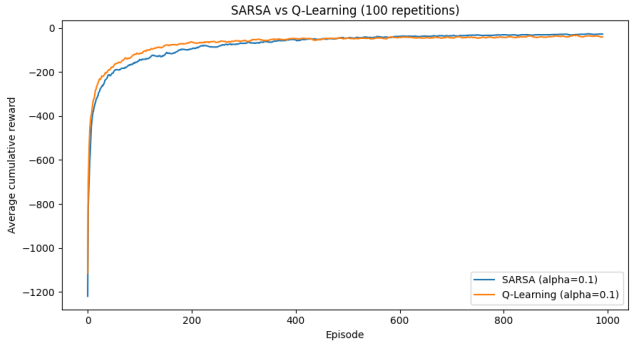


Figure 5: SARSA vs Q-Learning learning curves ($\alpha = 0.1$, $\varepsilon = 0.1$, 100 repetitions).

4 Stormy Weather: Windy Environment

4.1 Setup

In `WindyShortcutEnvironment`, after every action, there is a 50% probability that the agent is pushed one square downward, regardless of its intended action. This stochasticity means that even a perfectly greedy policy can end up in unexpected states, including the cliff.

4.2 Results and Interpretation

Q-Learning's greedy policy (Figure 7a) still approaches the cliff edge. Because its update rule assumes optimal future actions, it does not account for the possibility of being pushed off the cliff by the wind. SARSA's greedy policy (Figure 7b) is noticeably different: arrows point well away from the cliff, choosing a longer but safer route. Every time the wind pushed the agent onto the cliff during

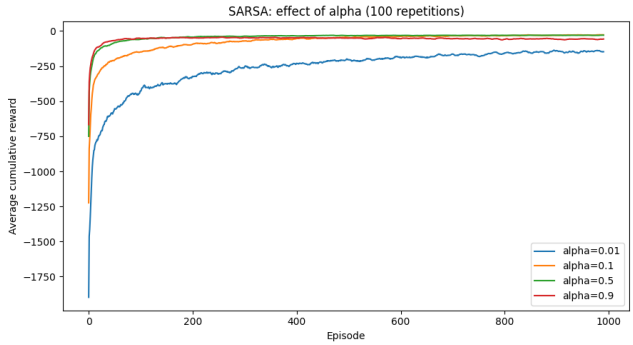
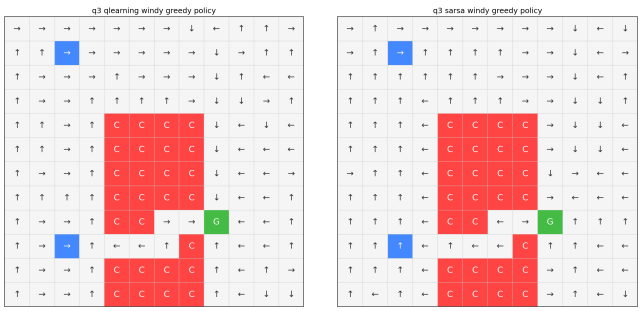


Figure 6: SARSA learning curves for $\alpha \in \{0.01, 0.1, 0.5, 0.9\}$, 100 repetitions.



(a) Q-Learning (windy). **(b) SARSA (windy).**

Figure 7: Greedy policies in the windy environment after 10 000 episodes ($\alpha = \varepsilon = 0.1$). Q-Learning stays near the cliff; SARSA avoids it.

training, SARSA reduced the Q-values of nearby states, and the policy gradually learned to maintain distance from dangerous areas.

The windy environment makes the on-policy versus off-policy distinction impossible to ignore. Q-Learning is blind to wind because its formula reasons only about what would happen under perfect future behavior. SARSA sees the wind's consequences directly, because it evaluates states using the policy actually being executed. If the wind came from a different direction — for instance, pushing the agent upward — the same logic would apply: SARSA would learn to avoid whichever states risk being pushed toward the cliff from the new angle.

5 Expected SARSA

5.1 Methodology

Expected SARSA sits conceptually between SARSA and Q-Learning. Plain SARSA samples the next action a' and uses $Q(s', a')$, introducing variance when the sampled action is unusually good or bad. Expected SARSA eliminates this variance by computing the exact expected Q-value over all possible next actions, weighted by their probability under the current ε-greedy policy. The update rule [1, Eq. 6.9] is:

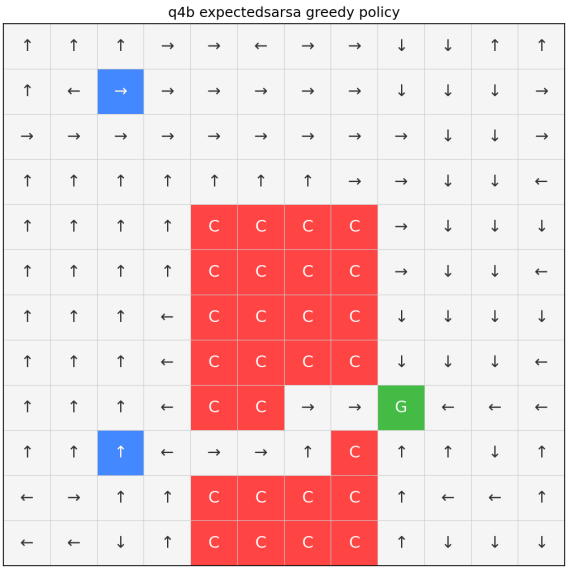


Figure 8: Greedy policy of Expected SARSA after 10 000 episodes.

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \sum_{a'} \pi(a'|s')\, Q(s',a') - Q(s,a)\Big] \quad (3)$$

Under ε-greedy, the greedy action receives probability $1-\varepsilon+\varepsilon/n$; every other action receives ε/n. The expected value is therefore mostly the best Q-value, with a small contribution from all other actions — more honest than Q-Learning's pure maximum and more stable than SARSA's random sample.

5.2 Results

The greedy policy (Figure 8) resembles SARSA's, avoiding the cliff region more carefully than Q-Learning. Because Expected SARSA is on-policy at heart, it incorporates the cost of exploration near dangerous states, though slightly less strongly than SARSA since the expectation dilutes the impact of any single random action.

The learning curve (Figure 9) sits between SARSA and Q-Learning, as theory predicts. Expected SARSA outperforms SARSA because lower-variance updates compound over thousands of steps into meaningfully better performance. The α comparison (Figure 10) shows that Expected SARSA handles larger α values slightly better than plain SARSA, since each update is already less noisy by design.

5.3 Interpretation

Expected SARSA demonstrates a general principle: when an exact calculation can replace a noisy estimate at low computational cost, it almost always should. SARSA and Expected SARSA are conceptually identical — both on-policy, both accounting for exploration risk — but Expected SARSA computes future values more carefully, producing a more reliable update signal at every step.

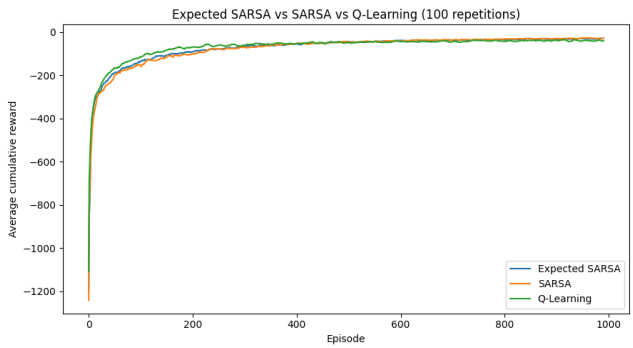


Figure 9: Expected SARSA vs SARSA vs Q-Learning ($\alpha = 0.1$, $\varepsilon = 0.1$, 100 repetitions).

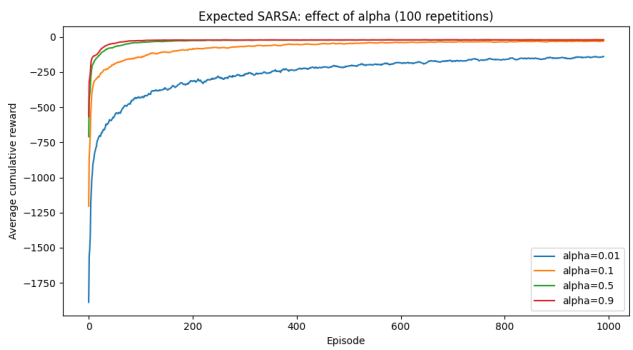


Figure 10: Expected SARSA learning curves for $\alpha \in \{0.01, 0.1, 0.5, 0.9\}$, 100 repetitions.

6 n-step SARSA

6.1 Methodology

All previous algorithms update from a single transition. n-step SARSA collects n real transitions before updating. The n-step return is:

$$G_{t:t+n} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n Q(s_{t+n}, a_{t+n}) \tag{4}$$

The update corrects $Q(s_t, a_t)$ toward this richer target:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[G_{t:t+n} - Q(s_t, a_t)\big] \tag{5}$$

When $n = 1$, Equation 5 reduces to standard SARSA. When n equals the full episode length, no bootstrapping occurs and the method becomes Monte Carlo. n-step SARSA is therefore a family parameterized by n, smoothly interpolating between SARSA and Monte Carlo.

6.2 Results

The greedy policy with $n = 5$ (Figure 11) resembles standard SARSA in its cliff-avoidance behavior. Multi-step returns propagate value information more efficiently: a good experience several steps from the goal directly updates states that are several steps earlier, rather than waiting for information to propagate one step at a time.

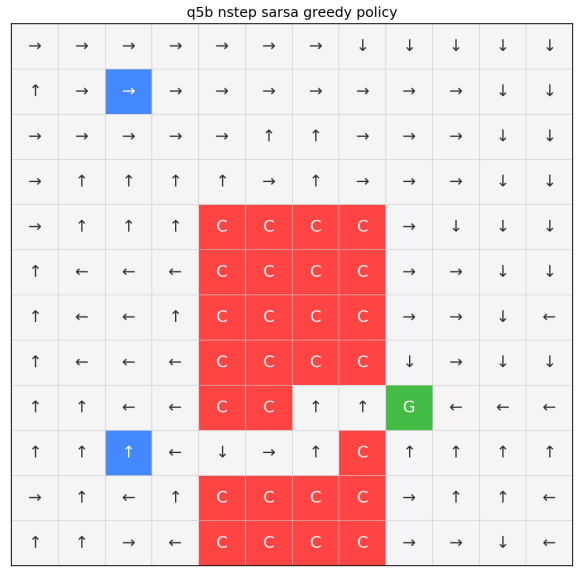


Figure 11: Greedy policy of n-step SARSA ($n = 5$) after 10 000 episodes.

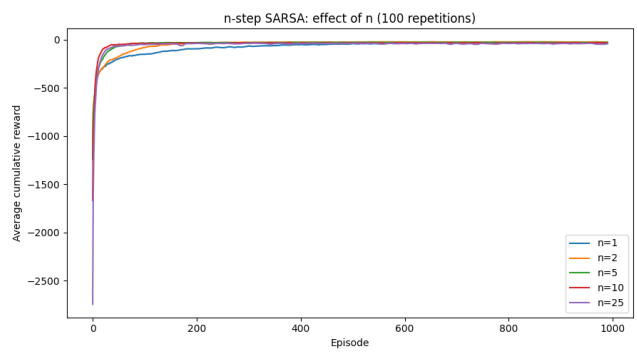


Figure 12: n-step SARSA for $n \in \{1, 2, 5, 10, 25\}$ ($\alpha = 0.1$, $\varepsilon = 0.1$, 100 repetitions).

Figure 12 shows the effect of $n \in \{1, 2, 5, 10, 25\}$ with $\alpha = 0.1$. Larger n leads to better performance within 1000 episodes: $n = 25$ achieves the highest reward, while $n = 1$ (standard SARSA) achieves the lowest.

6.3 Interpretation

The improvement from a larger n reflects how much real information each update carries. With $n = 1$, each update relies on one real reward plus a Q-value estimate that may be inaccurate early in training. With $n = 25$, 25 real rewards provide a strong foundation, making the update signal less dominated by estimation error. However, large n introduces a cost: the agent must wait n steps before its first update in each episode, and long trajectories produce higher-variance returns. This is the fundamental bias-variance trade-off of

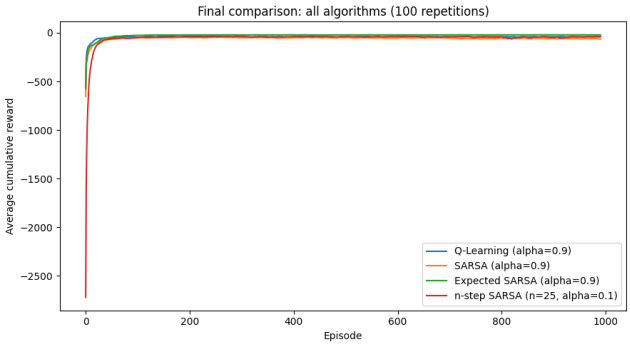


Figure 13: Final comparison: Q-Learning ($\alpha = 0.9$), SARSA ($\alpha = 0.9$), Expected SARSA ($\alpha = 0.9$), n-step SARSA ($n = 25$, $\alpha = 0.1$), 100 repetitions.

multi-step methods: small n introduces high bias (reliance on potentially inaccurate Q-estimates) but low variance; large n (Monte Carlo) eliminates bias but introduces high variance. For the ShortCut environment, $n = 25$ performs best because episodes are long enough to provide rich information without variance dominating. In environments with shorter episodes, the optimal n would likely be smaller.

7 Comparison and Conclusions

7.1 Quantitative comparison

Figure 13 shows all four algorithms at their best settings. All converge to similar final performance levels, reflecting that ShortCut is simple enough for all methods to solve. Primary differences lie in learning speed and convergence stability.

7.2 One-step algorithms

Q-Learning learns fastest at high α because the max operator extracts the most optimistic signal from each experience. It pays for this in two ways: the learned policy is riskier (close to the cliff), and it performs poorly in stochastic environments because it ignores actual transition randomness. SARSA is safer but slightly slower due to its conservative on-policy updates. Expected SARSA offers the best practical balance — on-policy like SARSA but with reduced variance from computing exact expected values. In most settings, Expected SARSA is a strictly better version of SARSA at minimal additional computational cost.

7.3 Single-step vs multi-step

n-step SARSA with $n = 25$ demonstrates that richer real experience can substitute for more clever bootstrapping. Rather than being smarter about estimating the future, it uses more of the actual past. This approach is powerful when episodes are long and exploration provides rich coverage. Its limitation is the need for careful joint tuning of n and α, and potential under performance in environments with very short episodes where large n cannot be fully utilized.

7.4 Reflection

The most important lesson from these experiments is that algorithm choice shapes what the agent actually learns to do. Q-Learning and SARSA produce visibly different policies even when their learning curves look similar. The on-policy versus off-policy distinction — a seemingly small mathematical change from $Q(s', a')$ to $\max Q(s', a')$ — translates directly into a safer versus riskier path in practice. This effect is invisible in the deterministic environment and becomes obvious in the windy one, a reminder that algorithms must always be evaluated in the context of the environment they will actually operate in.

A natural next step is to move beyond tabular methods. ShortCut has only 144 states, so a Q-table is feasible. In environments with large or continuous state spaces, a neural network must approximate $Q(s, a)$. The resulting deep Q-learning (DQN) applies exactly the same update rules studied here. All concepts encountered in this assignment — TD learning, on-policy versus off-policy, the bias-variance trade-off of multi-step returns, the role of α — are the direct theoretical foundations of modern deep reinforcement learning.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction.* MIT press, 2018.